Function capturing has multiple use cases. Let's look at how it can be used to reference a named function.

Capturing Named Functions

You just saw how you can create a new anonymous function using the & operator. But we can use this same technique to capture a named function. Capturing a named function lets us treat it like an anonymous function.

Named functions in Elixir are referred to by Module.name/arity. This same format is used to capture a named function. For example, the code my_fn = & IO.puts/1 assigns the IO.puts/1 function to the my_fn variable. You could then call it using my_fn.("hi"). Let's see this in action. The next example captures the IO.puts/1 function and uses it in an enumerator over a list:

```
$ iex
iex(1)> Enum.each([1, 2], & IO.puts/1)
1
2
:ok
```

Capturing is also possible in Ruby, but it looks slightly different because you use the method method:

```
$ irb
irb(main)> [1,2].each(&method(:puts))
1
2
=> [1, 2]
```

Elixir's function capturing helps you keep your code clean. If you have multi-line anonymous functions, it's often better to create a named function and use function capturing to reference it.

Let's go over an example of extracting an anonymous function into a named one. The following code has inline/0 and extracted/0 functions. inline/0 passes a multi-line anonymous function as an argument to Enum.each/2. But extracted/0 captures the print_name/1 function and passes that into Enum.each/2 instead.

elixir_examples/lib/examples/modules/extract.ex

```
defmodule Examples.Modules.Extract do
  def inline do
    Enum.each(names(), fn name ->
      capitalized = String.capitalize(name)
      phrase = "Hello, #{capitalized}"
      IO.puts(phrase)
    end)
  end
```